

Design and Implementation of a Control Agent for a Manufacturing Plant

Lecture: Discrete Simulation and Reinforcement Learning

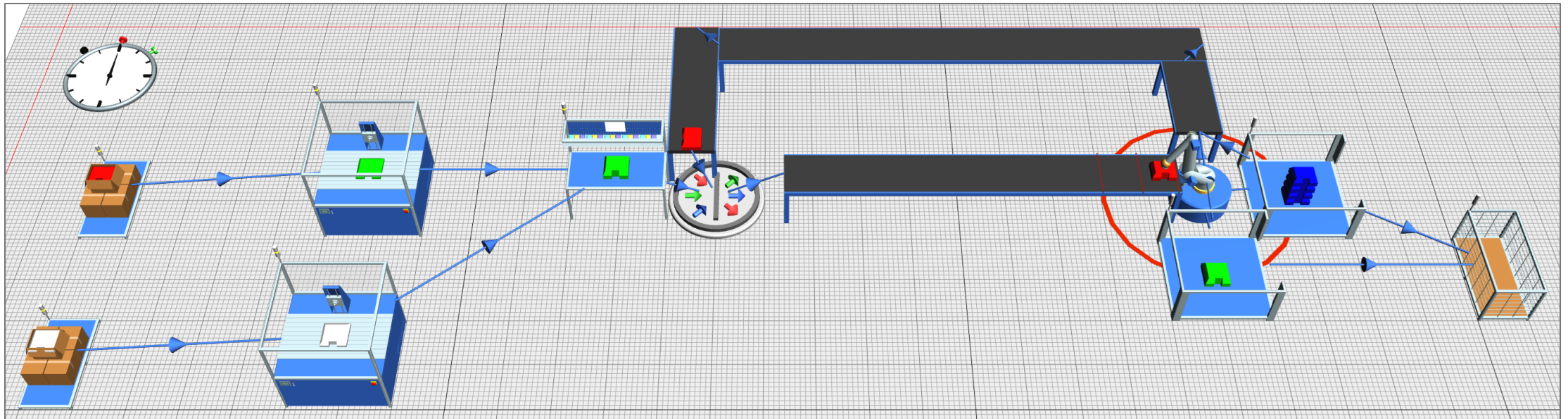
Prof. Dr. Jonas Ide and Prof. Dr. Christian Schwede

Name: Niko Kläsener

Matr.-Nr: 1314104

Project Description

- Autonomous RL agent controlling a Pick-and-Place robot
- Sorting 3 language variants using 2 limited buffers
- Minimizing total time for 1,000 completed games



PEAS framework

Dimension	Specification
Performance (P)	Minimize throughput time and operational steps for 1,000 games.
Environment (E)	▪ Discrete-event simulation ▪ deterministic mechanics ▪ stochastic arrivals ▪ fully observable
Actuators (A)	Pick-and-place robot: ▪ Buffer 1 ▪ Buffer 2 ▪ Return loop
Sensors (S)	▪ Incoming workpiece type ▪ buffer workpiece types ▪ buffer filllevels

RL approach Q-Learning Agent

Decision	Reason
Modelfree TD-learning	deterministic mechanics
Manageable Q-table dimension	- States: incTyp[3]; buffer1Typ[4]; buffer2Typ[4]; buffer1cnt[10]; buffer2cnt[10] - Action: buffer1; buffer2; returnloop - Dimension: 4800x3
GLIE scheme	exploration

Advantages compared to Deep Q-Learning:

- Convergence guarantee for finite MDPs
- No forgetting of past conditions
- 100% transparency

Reward-Design

$$R_{(s,a,s')} = -1 + \begin{cases} n & \text{for matching store (fill level } n < 10) \\ 100 & \text{for 10th matching part (batch drain)} \\ (-n) & \text{for mismatch part (flushes } n \text{ parts)} \end{cases}$$

fixed penalty for
each required step

Individual
reward/punishment for the
outcome of an action

Parameterization TD and GLIE

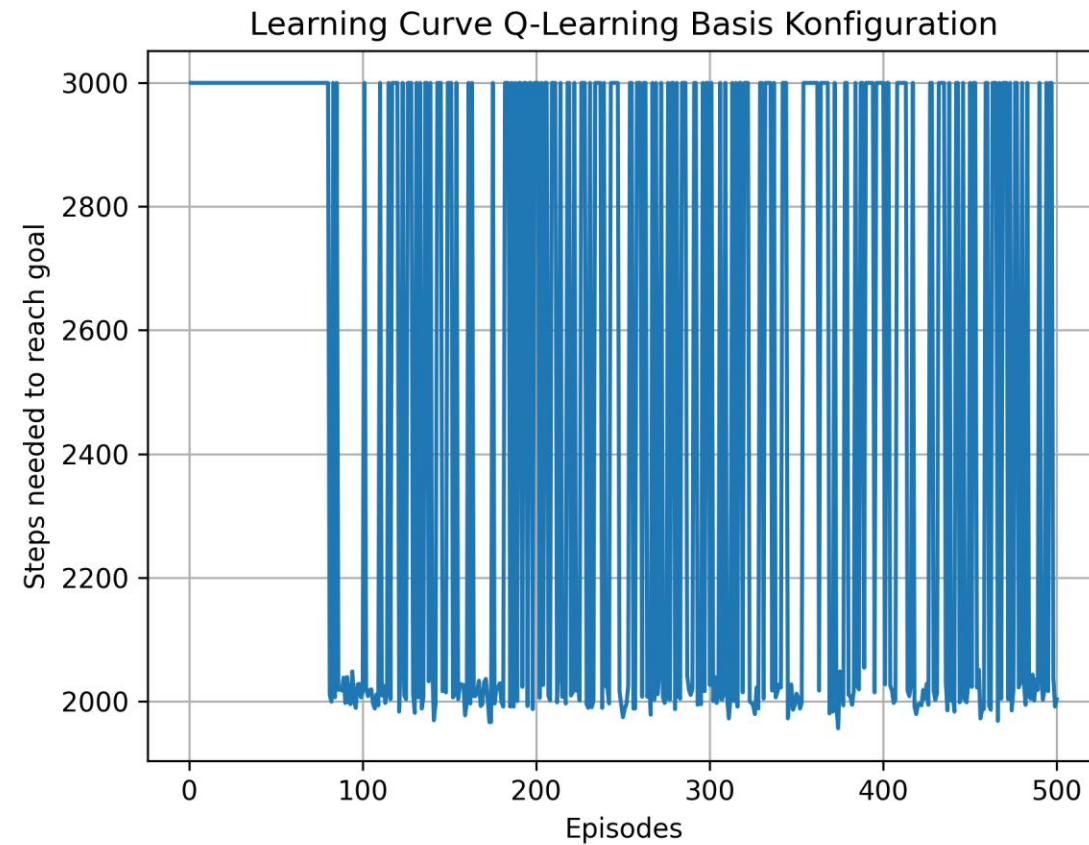
Temporal-difference learning		GLIE Scheme	
$U_{(s)}^{\pi} = U_{(s)}^{\pi} * \alpha[R_{(s,\pi(s),s')} + \gamma U_{(s')}^{\pi} - U_{(s)}^{\pi}]$		$f_{(u,n)} = \begin{cases} R^+ & n < N_e \\ u & \text{else} \end{cases}$	
		$a = \max f_{(Q(s,a),N(s,a))}$	
Learning rate	$\alpha = 0.1$	Optimism under Uncertainty	$R^+ = 2000$
Discount Factor	$\gamma = 0.99$	Max exploration	$N_e = 10$

Learning Basic Configuration

State Design:

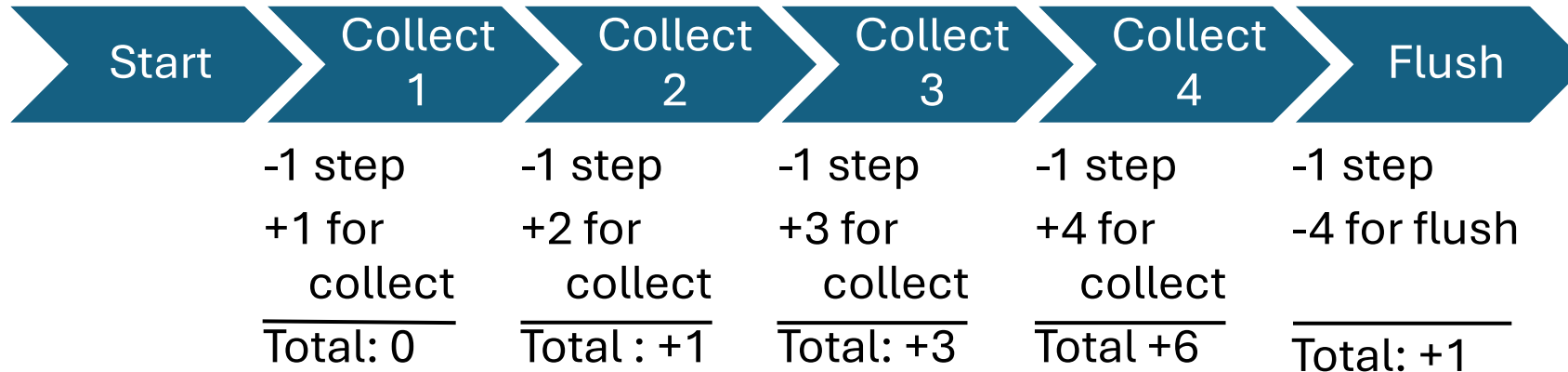
- incTyp: 3 [EN; DE; ES]
- Buffer1Typ: 4 [empty; EN; DE; ES]
- Buffer2Typ: 4 [empty; EN; DE; ES]
- Buffer1cnt: 3 [(0-2); (3-6); (7-9)]
- Buffer2cnt: 3 [(0-2); (3-6); (7-9)]

Learning status	Number of states	Percent
Fully explored	216	72%
Partially explored	52	17,3%
Not explored	32	10,7%
physically possible	300	100%



Reward farming

State Index	Incoming Part	Buffer 1 (Type / Fill)	Buffer 2 (Type / Fill)	Q(Buffer 1)	Q(Buffer 2)	Q(Return)	Selected Action	Evaluation
60	DE	DE (7-9)	EN (0-3)	575.69	-0.69	-0.20	Buffer 1 (A0)	Optimal
61	DE	DE (7-9)	EN (4-6)	2.14	543.96	0.25	Buffer 2 (A1)	Reward Farming
62	DE	DE (7-9)	EN (7-9)	580.68	1.42	-0.20	Buffer 1 (A0)	Optimal
353	ES	DE (0-3)	ES (7-9)	611.01	9.59	0.66	Buffer 1 (A0)	Reward Farming
356	ES	DE (4-6)	ES (7-9)	607.73	2.05	1.05	Buffer 1 (A0)	Reward Farming
359	ES	DE (7-9)	ES (7-9)	598.11	13.44	4.87	Buffer 1 (A0)	Reward Farming



$$R_{(s,a,s')} = -1 + \begin{cases} n \\ 100 \\ (-n) \end{cases}$$

New Reward-Design

$$R_{(s,a,s')} = -1 + \begin{cases} n & \text{for matching store (fill level } n < 10) \\ 100 & \text{for 10th matching part (batch drain)} \\ \frac{(1+n)*n}{2} & \text{for mismatch part (flushes } n \text{ parts)} \end{cases}$$

Diagram illustrating the components of the reward function $R_{(s,a,s')}$:

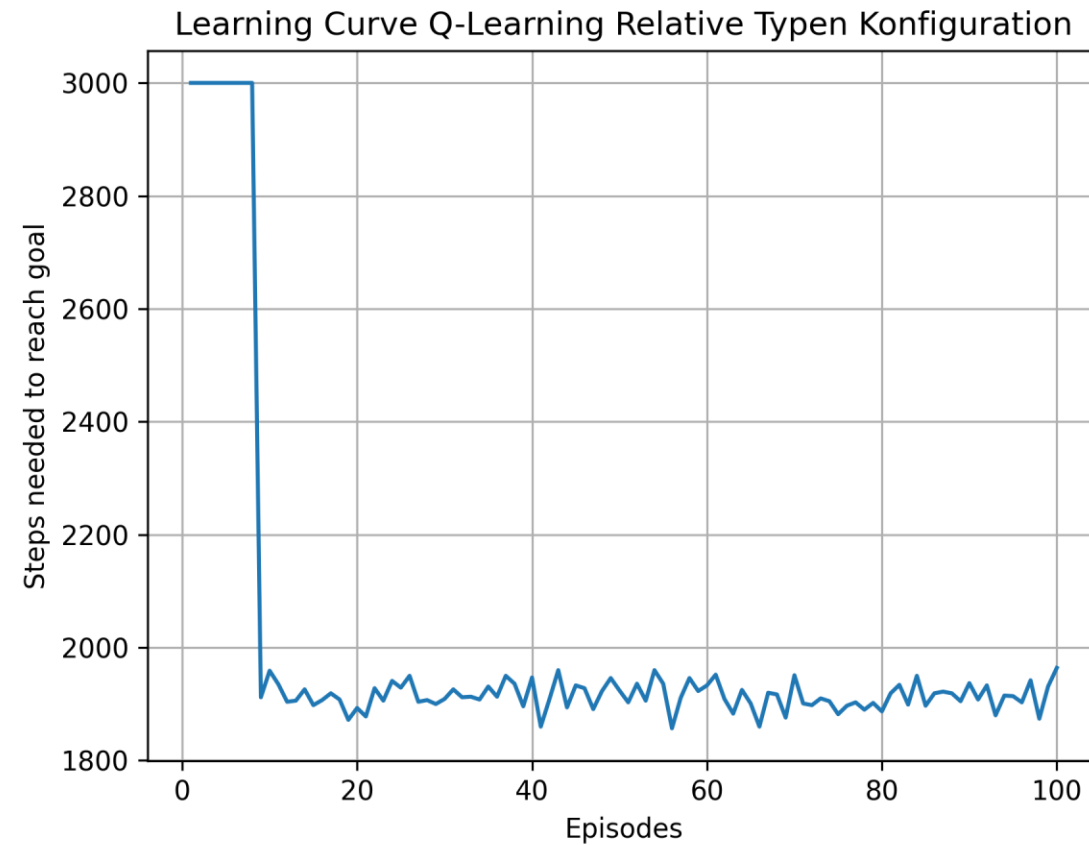
- fixed penalty for each required step**: Points to the -1 term.
- Individual reward/punishment for the outcome of an action**: Points to the summation term in the curly braces.
- Gauss's summation formula**: Points to the formula $\frac{(1+n)*n}{2}$ used for mismatch parts.

Learning Relative State Configuration

State Design:

- Buffer1Typ: 3 [empty; match; mismatch]
- Buffer2Typ: 3 [empty; match; mismatch]
- Buffer1cnt: 3 [(0-2); (3-6); (7-9)]
- Buffer2cnt: 3 [(0-2); (3-6); (7-9)]

Learning status	Number of states	Percent
Fully explored	49	100%
Partially explored	0	0%
Not explored	0	0%
physically possible	49	100%



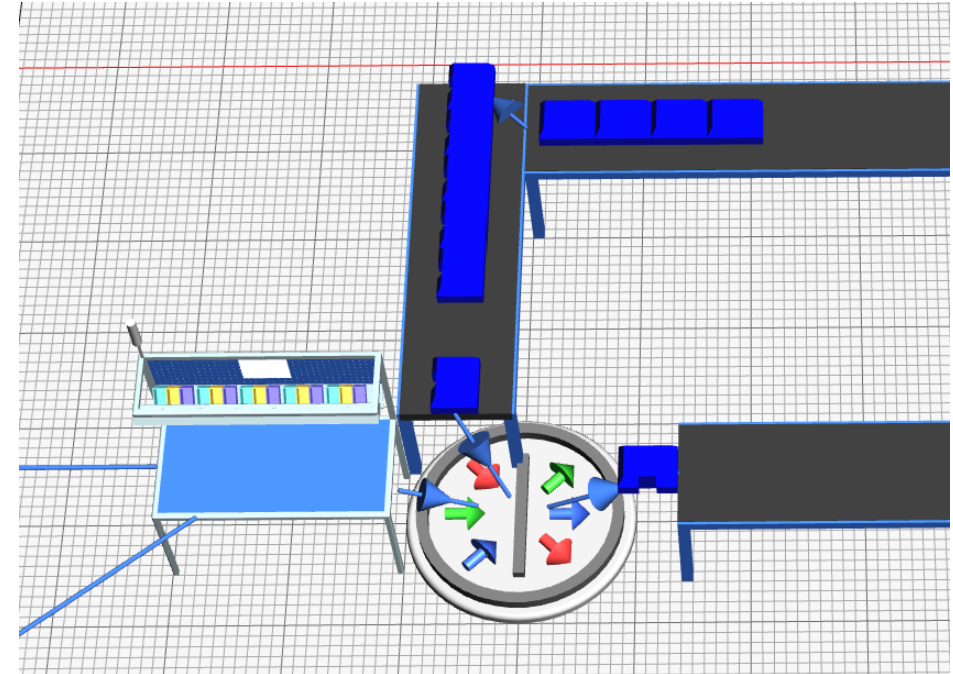
Memory State

Items not currently being collected accumulate on the return loop.

Once the agent switches to this type after flushing a buffer, it can refill empty buffers rapidly.

State LastCollected:

- keeps track of which type was collected last
- 3 [Warm-Up, Collected, NotCollected]

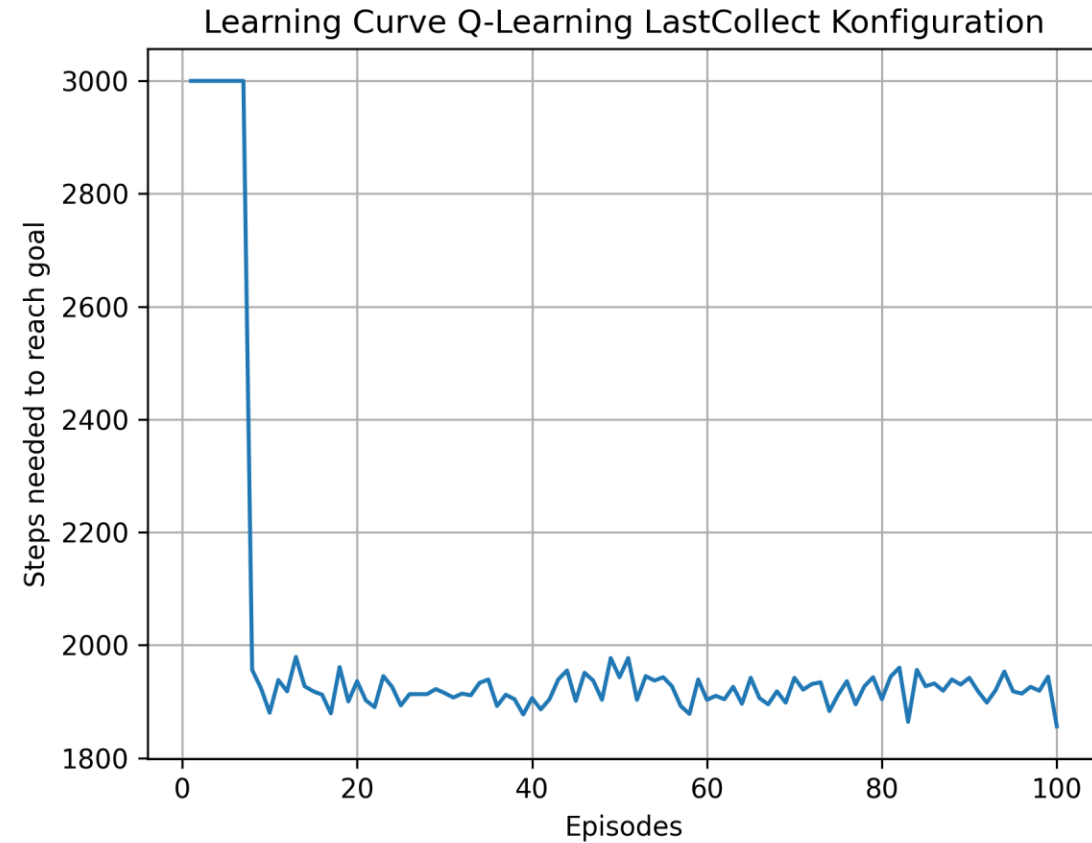


Learning Last Collected Configuration

State Design:

- Buffer1Typ: 3 [empty; match; mismatch]
- Buffer2Typ: 3 [empty; match; mismatch]
- Buffer1cnt: 3 [(0-2); (3-6); (7-9)]
- Buffer2cnt: 3 [(0-2); (3-6); (7-9)]
- LastCollected: 3 [Warm-Up, Collected, NotCollected]

Learning status	Number of states	Percent
Fully explored	51	89%
Partially explored	2	4%
Not explored	4	7%
physically possible	57	100%



Analysed Last Collected Configuration

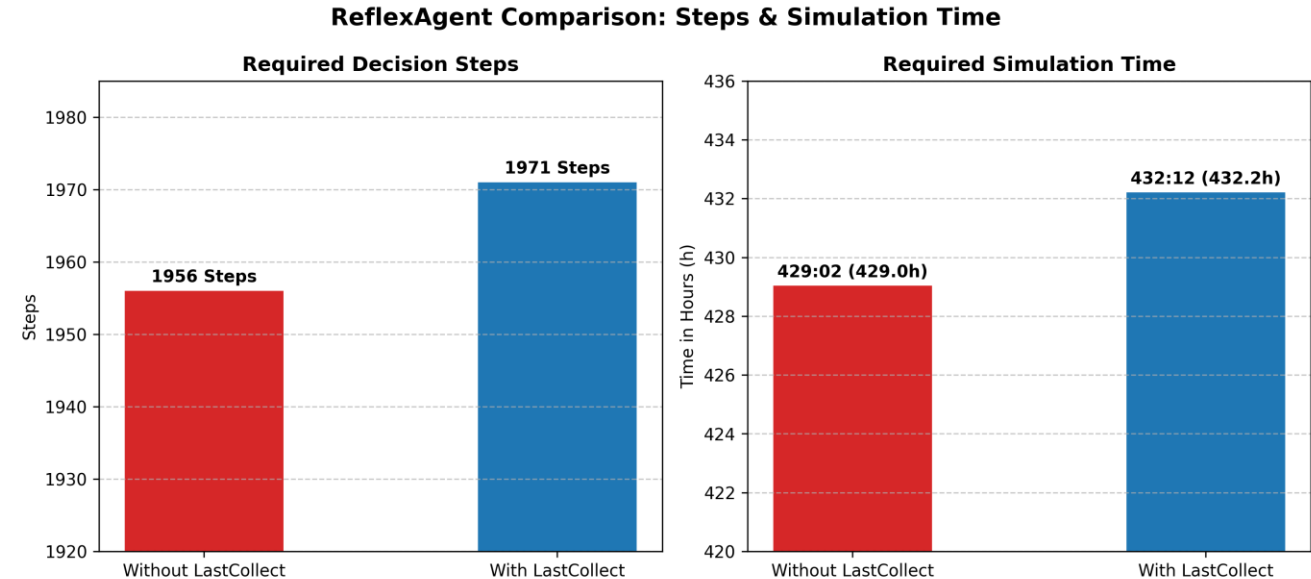
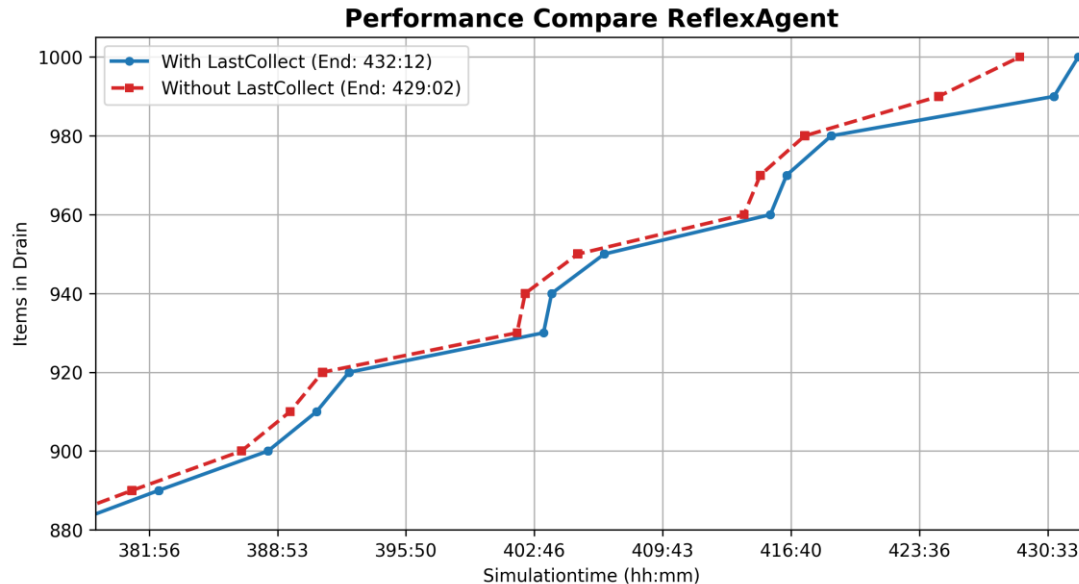
State Index	Buffer 1 (Rel. / Fill)	Buffer 2 (Rel. / Fill)	LastCollect (Memory)	Q(Buffer 1)	Q(Buffer 2)	Q(Return Loop)	Selected Action	Evaluation
55	Empty	Dismatch (0-2)	Collected (1)	604.56	-1.15	-0.27	Buffer 1 (A0)	Send to empty buffer
56	Empty	Dismatch (0-2)	NotCollected (2)	598.77	-1.04	-0.28	Buffer 1 (A0)	Send to empty buffer
58	Empty	Dismatch (3-6)	Collected (1)	637.18	-3.98	-0.17	Buffer 1 (A0)	Send to empty buffer
59	Empty	Dismatch (3-6)	NotCollected (2)	637.04	-4.33	-0.30	Buffer 1 (A0)	Send to empty buffer

The agent does not differentiate between a part that was collected last and one that was not collected last.

Validation Last Collected learning

Policy comparison (Simple Reflex Agents):

- **Agent 1:** Immediately fills any free buffer.
- **Agent 2:** Waits for a game different from the last collected.



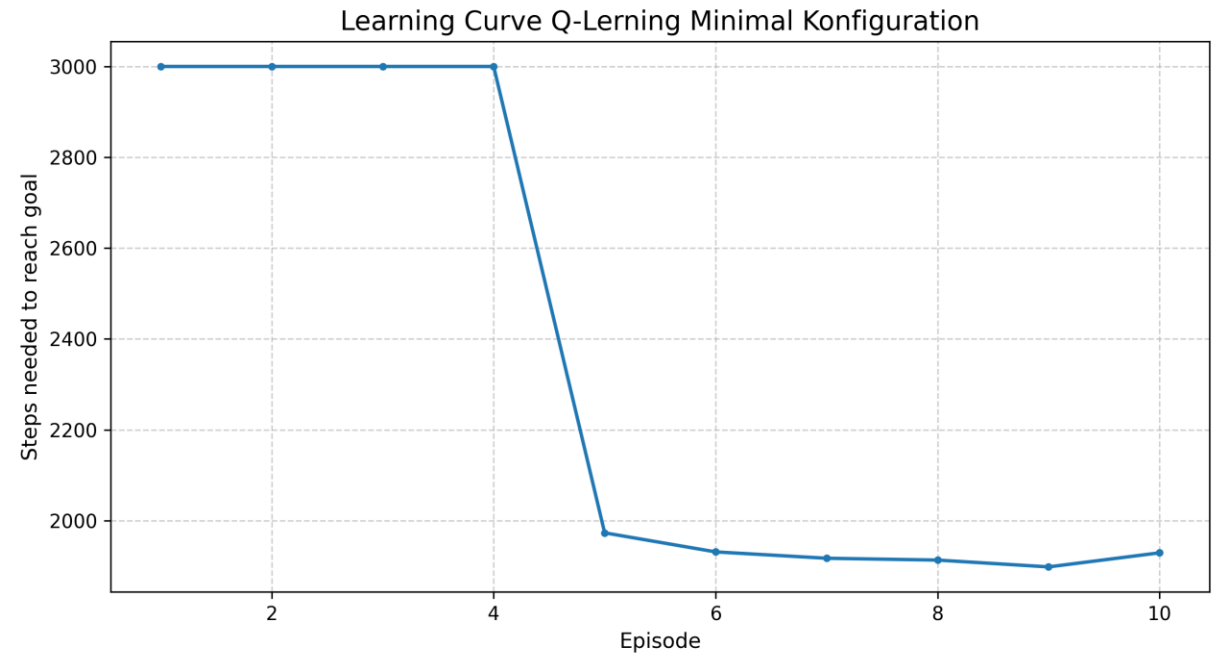
Waiting for a non-lastcollect type only increases runtime. Ignoring lastcollect during Q-learning is therefore optimal, not a flaw.

Learning Minimal Configuration

State Design:

- Buffer1Typ: 3 [empty; match; mismatch]
- Buffer2Typ: 3 [empty; match; mismatch]

Learning status	Number of states	Percent
Fully explored	9	100%
Partially explored	0	0%
Not explored	0	0%
physically possible	9	100%

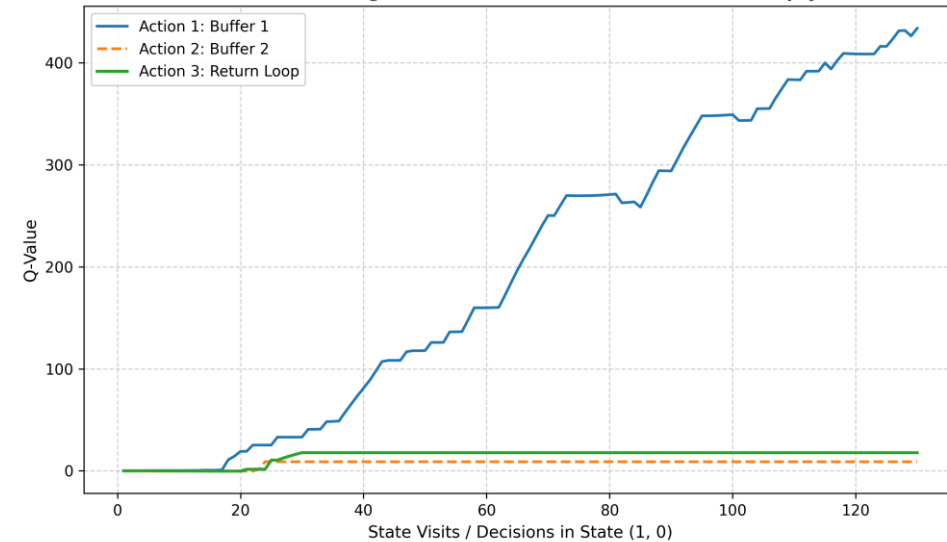


Learning Minimal Configuration

Trained Q-Table (9 States x 3 Actions)

State Index	Buffer 1	Buffer 2	Q(Buffer 1)	Q(Buffer 2)	Q(Return Loop)	Best Action
0	Empty (0)	Empty (0)	0.01	303.26	-0.61	Buffer 2 (2)
1	Empty (0)	Match (1)	2.79	461.41	3.29	Buffer 2 (2)
2	Empty (0)	Dismatch (2)	510.15	-1.90	-0.67	Buffer 1 (1)
3	Match (1)	Empty (0)	433.69	8.93	17.79	Buffer 1 (1)
4	Match (1)	Match (1)	9.84	102.01	6.19	Buffer 2 (2)
5	Match (1)	Dismatch (2)	524.40	-1.57	-0.56	Buffer 1 (1)
6	Dismatch (2)	Empty (0)	-1.54	509.94	-0.63	Buffer 2 (2)
7	Dismatch (2)	Match (1)	-8.76	533.38	-0.62	Buffer 2 (2)
8	Dismatch (2)	Dismatch (2)	-1.85	-2.05	512.00	Return (3)

Q-Value Convergence for State (1, 0) [B1=Match, B2=Empty]



Conclusion & Key Takeaways

Key Findings & Optimization

- **Best Performance with Lean State:** Relative states (match / mismatch) achieve the best result of **1,850–1,950 steps / 1,000 games**.
- **History Adds No Value:** Tracking lastcollect increases state size without making the robot faster.
- **Main Takeaway:** Smart feature design beats brute-force state expansion.

Limits & Future Work

- **Limit Reached:** The strategy is already optimal for the current setup.
- **Further Speedups Need More Degrees of Freedom:** Reducing steps further requires additional actions for the agent:
 - Priority control between the main line and return loop.
 - Dynamic conveyor belt speeds.